

Push Your Agent: Measuring and Enforcing Quantitative Goal Persistence in Long-Horizon LLM Agents

Yuandao Cai¹ Yuzhang Zhu¹ Liyou Gao¹ Wensheng Tang¹ Shengchao Qin²

¹Independent Researcher ²Xidian University

Abstract

Long-horizon language agents can make many plausible local tool calls yet fail to persist until a requested count is actually complete. We study this gap as Quantitative Goal Persistence (QGP): whether an agent keeps working until an external verifier confirms enough distinct valid items. PushBench turns this into a benchmark for repository-artifact collection and verifier-backed work units, so repeated work, duplicate submissions, false completion, and progress drift are measured directly rather than hidden behind a final success flag. In matched controller comparisons, a state-tracking retrieval controller reaches 69–78% success while eliminating duplicate submissions, and a backlog-tracking work-unit controller reaches 25–50% success in settings where standard and completion-gated controllers complete no task instances. Black-box frontier-agent evaluations with Claude Code (Sonnet 4.6) and Codex CLI (gpt-5.4) solve many 50-artifact tasks but drop to 3/9 successes per condition at 100 artifacts. The results show that quantitative goals stress a different reliability requirement from local task competence: agents must maintain verified progress and stop only when the requested work is complete.

1 Introduction

Tool-using language agents are now routinely asked to search, collect evidence, edit code, operate web interfaces, and coordinate many steps before producing an answer. Many such tasks are not complete when the agent has found *some* useful items or made *one* plausible edit. They are complete only when an explicit, auditable condition has been met: a backlog cleared, required records processed, failing checks passed, or enough repository artifacts collected. In these settings, local competence is not enough. The reliability problem is whether an agent’s progress toward an explicit goal is maintained until the completion condition is satisfied.

Recent agent benchmarks cover increasingly realistic reasoning-and-acting frameworks, web environments, and repository tasks (Yao et al., 2023; Schick et al., 2023; Yao et al., 2022; Zhou et al., 2024; Jimenez et al., 2024). They capture task success across interactive settings (Liu et al., 2023; Xie et al., 2024; Ma et al., 2024a; Yao et al., 2024). However, final success rates alone often hide how progress was lost. A failed trajectory may reflect poor local decisions, but it may also reflect premature stopping, repeated work, duplicate submissions, or a final message that overstates how much verified work has actually been completed. This is especially problematic when an agent appears helpful for many turns and still stops short of the requested count.

We study this behavior through Quantitative Goal Persistence (QGP), which asks whether an agent continues until an external verifier confirms at least N distinct valid work units. This framing makes duplicate submissions, false completion, premature stopping, and reported-count error measured failure modes rather than incidental artifacts of failed trajectories. QGP therefore differs from final success, $\text{recall}@N$, or ordinary pass/fail scoring. It also differs from progress-oriented evaluations that assign partial credit to trajectories (Ma et al., 2024a): QGP makes the stopping condition itself externally checkable, so repeated submissions, premature stopping, and unsupported completion claims are part of the task outcome.

PushBench is a benchmark and evaluation framework for this kind of verifier-audited progress. It contains two controlled task families. Repository-artifact collection (QGP-RepoScan) isolates counting and retrieval over real repository artifacts. Verifier-backed work units (QGP-DataOps-lite) extend the setting to small edits, checks, and structured updates that count only after a deterministic verifier accepts them. These controlled, verifier-checkable, low-to-medium difficulty tasks reduce

impossible-goal explanations and more directly measure quantitative persistence. Note that we also include coding-style and black-box frontier-agent evaluations to test whether the same progress checks remain useful in more realistic settings, where persistence is intertwined with coding skill, repository navigation, hidden prompting, tool routing, and product-specific loops. Figure 1 gives the workflow: task definitions provide the goal and valid-unit criteria, while the controller and verifier maintain progress through repeated actions.

We further evaluate controller-level interventions tailored to the two PushBench task families. For repository-artifact retrieval, a state-tracking retrieval controller (STATEQGP) keeps externally visible state over submitted identifiers, repeated search pages, and completion eligibility. For verifier-backed work units, a backlog-tracking controller (UNITQGP) keeps verifier-visible state over pending, attempted, and passed units. These controllers do not alter the verifier or the set of valid items; they change how verified progress is exposed, tracked, and enforced so that an execution cannot repeatedly spend budget on completed or no-progress work.

Across these settings, progress-state mechanisms reduce repeated work and premature stopping. In repository-artifact collection, state-tracking control reaches 69–78% success under matched model and backend settings while eliminating duplicate submissions. In verifier-backed work-unit tasks, standard and completion-gated controllers complete no task instances in this evaluation, while backlog-tracking control reaches 25–50% success. Memory agents and frontier systems can help, but they do not make verified progress automatic: black-box frontier-agent evaluations solve many 50-artifact tasks but fall sharply at 100 artifacts. Overall, QGP failures remain visible even when stronger models, memory mechanisms, and frontier-agent harnesses improve absolute performance. Our contributions are:

- We formalize QGP as an evaluation target for auditable progress toward explicit count goals.
- We design PushBench with repository-artifact and verifier-backed work-unit task families.
- We evaluate controller-level interventions under matched backends against passive, verifier-gated, and memory-agent baselines, with coding-style and black-box frontier-agent

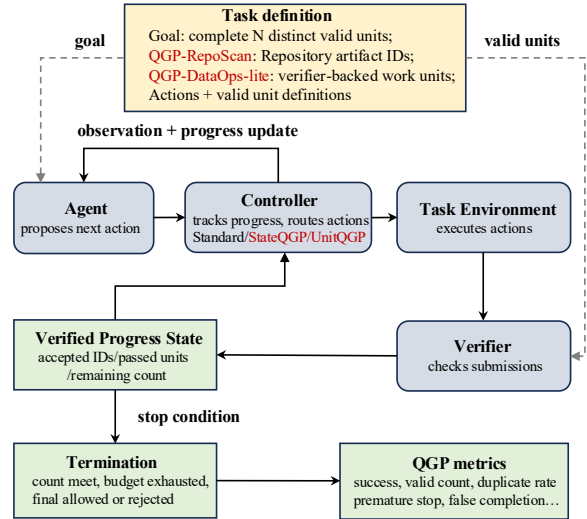


Figure 1: PushBench workflow: agents act through a controller, task environment, and verifier until the count goal is met or the budget is exhausted.

evaluations testing how the pattern transfers beyond the controlled settings.

2 Related Work

Agent and software benchmarks. Recent benchmarks evaluate agents in interactive web, desktop, data, and tool-using environments (Liu et al., 2023; Yao et al., 2022; Zhou et al., 2024; Xie et al., 2024; Yoran et al., 2024; Yao et al., 2024; Ma et al., 2026, 2024b); related text-environment and web navigation tasks include ALFWorld, ScienceWorld, and Mind2Web (Shridhar et al., 2021; Wang et al., 2022; Deng et al., 2023). Software benchmarks similarly cover code generation, repository context, and issue repair, from HumanEval, MBPP, CodeXGLUE, and APPS to InterCode, RepoBench, SWE-bench, and SWE-agent (Chen et al., 2021; Austin et al., 2021; Lu et al., 2021; Hendrycks et al., 2021; Yang et al., 2023; Liu et al., 2024; Jimenez et al., 2024; Yang et al., 2024). These benchmarks establish task realism and executable success; PushBench instead isolates quantitative persistence and externally auditable stopping conditions.

Tool use and agent control. Work on tool use and control improves how language models select tools, interleave reasoning and action, and recover from intermediate mistakes. ReAct, Toolformer, ToolLLM, API-Bank, Gorilla, StableToolBench, ToolHop, and tool-use alignment benchmarks study tool invocation and API use (Schick et al., 2023; Qin et al., 2024; Li et al., 2023; Patil et al., 2024;

Guo et al., 2024; Ye et al., 2025; Chen et al., 2024; Yao et al., 2023); middleware, model-based planning, reflection, and self-feedback work study execution infrastructure and trajectory repair (Gu et al., 2024, 2025; Shinn et al., 2023; Madaan et al., 2023). These methods can improve local action choice and recovery, but they typically leave the quantitative stopping invariant implicit. PushBench studies a different layer: the controller contract between policy actions and verifier-visible progress state.

Progress, partial credit, and trajectory evaluation. AgentBoard proposes fine-grained progress rates for multi-turn agents (Ma et al., 2024a), and interactive benchmarks often report partial completion or trajectory-derived signals. QGP differs in making the stopping condition itself the object of evaluation: false completion, premature stopping, duplicate work, and reported-count error are failure modes that show whether externally auditable progress has been maintained. This makes verified progress tracking part of the task contract rather than a post-hoc trajectory summary.

Memory and orchestration. Memory-augmented and long-running agents manage context, user preferences, and evolving state across interactions, including explicit memory systems, persistent simulated or embodied agents, and general autonomous-agent architectures (Packer et al., 2023; Park et al., 2023; Wang et al., 2024a,b). These systems are natural comparisons because QGP failures can look like memory failures. Our distinction is that QGP state is not arbitrary recalled context: it is externally checkable progress over submitted units, duplicates, remaining target count, and termination eligibility.

3 Quantitative Goal Persistence

We define a QGP task as a tuple $(\mathcal{E}, T, V, N, B)$, where $\mathcal{E}$ is an interactive environment, T is the task objective, $V(x) \in \{0, 1\}$ is an external verifier for a submitted work unit x , N is the target count, and B is the interaction budget. A work unit can be any verifier-visible item whose identity and validity can be checked, such as a repository backlog item, issue, data record, file chunk, code location, test case, build target, or queue item. At step t , the agent may inspect the environment, attempt a work unit, submit an identifier or artifact, ask the user, or produce a final answer. Let C_t be the multiset of submitted or attempted work-unit identifiers and

let D_t be the set of distinct identifiers in C_t . The verified progress at step t is

$$\text{valid_count}_t = |\{x \in D_t : V(x) = 1\}|. \quad (1)$$

The run is complete if and only if

$$\text{valid_count}_t \geq \text{target_count} = N. \quad (2)$$

Completion fidelity. An agent may report a count `reported_countt` or claim completion. We define a *false completion* as a final action with a completion claim when `valid_countt` $< N$. We define *premature stopping* as an ask-user or non-completing final action before `valid_countt` $\geq N$. The normalized reported-count error for a run with a reported count is

$$\frac{|\text{reported_count} - \text{valid_count}|}{\max(1, N)}. \quad (3)$$

Progress inflation occurs when `reported_count` $>$ `valid_count`.

Repeated work. Because long-horizon agents often reprocess the same evidence or retry the same failed item, we track repeated work separately from invalid work. In identifier-retrieval tasks, repeated work appears as duplicate submissions. In back-log/verifier tasks, it appears as repeated attempts on an already-passed unit or repeated failed attempts that do not change verified progress. This metric captures wasted interaction budget and loss of external memory.

Verifier-gated completion. A controller is verifier-gated if it intercepts final or ask-user actions and refuses to terminate the run while `valid_count` $< N$. Verifier gating does not make the underlying policy more capable at editing code, ranking evidence, or identifying relevant items. It changes the execution contract so that unsupported completion claims cannot end the interaction.

4 Benchmark Design

PushBench implements QGP through verifier-checkable task families in which each task specifies an objective, a target count, a budget, and a verifier-derived success condition. Agents emit structured actions, and outcomes are computed from verifier-visible state rather than final-message claims. The workflow in Figure 1 separates task definition, action execution, verification, and metric computation. Because every accepted item is recorded by

the verifier, the benchmark can separate target completion from false completion, duplicate submission, reported-count error, and premature stopping.

The controlled task families intentionally keep unit difficulty low-to-medium: if the units themselves require open-ended coding or domain expertise, a failure may reflect unsolved work rather than lost quantitative persistence. This design reduces that confound so repeated work, stale progress state, and unsupported completion claims can be attributed directly to QGP failures.

QGP-RepoScan task family. QGP-RepoScan is a repository-scanning benchmark family. In our evaluation, task instances are generated from local snapshots of requests, pytest, and flask (Python Software Foundation, 2026; pytest-dev, 2026; Pallets, 2026). During an interaction, the agent searches repository-derived candidates and submits stable artifact identifiers. A task succeeds only when the hidden verifier accepts at least N distinct submitted identifiers. Thus QGP-RepoScan tests whether an agent maintains verifier-visible progress across many search-and-submit steps.

QGP-DataOps-lite task family. QGP-DataOps-lite extends QGP from identifier retrieval to verifier-backed work units. Each task instance receives a backlog drawn from public data snippets, including Vega cars data (Vega Project, 2026) and FiveThirtyEight airline-safety data (FiveThirtyEight, 2026), plus repository-derived fixtures built from the same requests, pytest, and flask snapshots used by QGP-RepoScan. These fixtures turn repository files and metadata into small checker-backed units, such as CSV-field checks, file-metadata repairs, or consistency answers. A unit counts only after its checker accepts it, and the task succeeds only when at least N distinct units pass.

The two families are complementary. QGP-RepoScan isolates persistence in a search-and-submit setting where valid work is a distinct repository identifier. QGP-DataOps-lite keeps the same count invariant but adds short work loops: inspecting an artifact, producing or editing an answer, and submitting the unit only after a checker accepts it. This progression checks whether QGP failures persist when the agent must do small pieces of work rather than only retrieve identifiers.

Policies and harnesses. PushBench separates the policy that proposes work from the controller that executes or audits it. Native LLMPolicy and

LangGraph-backed policies are controlled execution backends rather than representative samples of all agent architectures. They share task manifests, budgets, verifiers, hidden valid sets, and summary fields, letting controller comparisons isolate changes in state exposure, duplicate filtering, completion gating, and repair routing. The experimental setup below specifies target counts, budgets, and aggregation units for each family. Additional task-family construction details are in Appendix A.

5 Persistence Controllers

PushBench distinguishes passive, verifier-gated, and stateful persistence controllers. Across all modes, hidden valid identifiers and labels remain inside the verifier. Controllers observe only the feedback produced for submitted units, such as accept/reject decisions, duplicate status, verified counts, and remaining count.

STATEQGP for identifier retrieval. STATEQGP is a controller-level mechanism for search-and-submit tasks. The policy still chooses actions, but the controller stores submitted identifiers, seen search pages, the last query, and the next unseen page for each query. When the policy repeats a search page, the controller advances to the next unseen page. When the policy submits identifiers, the controller filters identifiers that were already submitted or repeated within the same action. If filtering leaves an empty submission, the controller can repair the action into a search for the next unseen page. When the policy asks the user or claims completion before $\text{valid_count} \geq N$, the controller blocks the termination and returns a verifier observation explaining that the target has not yet been met.

Standard and verifier-gated controls. The standard controller executes parsed policy actions without duplicate-aware persistence repair. The verifier-gated controller adds completion gating: it blocks unsupported final or ask-user termination while $\text{valid_count} < N$. This isolates whether preventing false completion is sufficient without adding stronger state over already-submitted artifacts.

UNITQGP for verifier-backed units. UNITQGP extends the same idea to backlog tasks. It tracks unit status, notices stale inspection or no-submit loops, steers the policy back to the first pending or attempted unit, repairs post-edit behavior toward verifier execution, and

records verifier recoveries when a later repair turns an initially failed or incomplete trajectory into a passing unit. As with STATEQGP, the reported behavior includes visible controller assistance.

These controllers make a narrow claim: external state can enforce persistence invariants that are difficult to leave entirely to model memory. PushBench reports passive and controller-assisted execution separately to keep this assistance visible.

6 Experimental Setup

Experiment 1 evaluates QGP-RepoScan. The QGP-RepoScan manifest has 36 task instances generated from local snapshots of requests, pytest, and flask. Target counts are $N \in \{10, 25, 50, 100\}$, with nine task instances per target. Predicate families include keyword-or-pattern, path-and-content, and test-or-documentation artifacts. The success invariant is $\text{valid_count} \geq \text{target_count}$ over unique repository artifact identifiers.

Experiment 2 evaluates QGP-DataOps-lite. Each task instance uses a backlog of public-data and repository-derived fixture work units with unit verifiers. Targets are $N \in \{3, 5, 10, 20\}$, with six backlogs per target. The evaluation crosses two agent implementations, three controllers, four target counts, and six seeds or backlogs per target. Success again requires $\text{valid_count} \geq \text{target_count}$, now over verifier-passing work units rather than retrieved artifact IDs.

We use *task instance* to mean one fixed manifest or backlog, and *run* to mean one policy-controller execution on one task instance. Thus a QGP-RepoScan row with 36 executions covers four target counts with nine task instances each, while a QGP-DataOps-lite cell with 24 executions covers four target counts with six backlogs each. These controlled, verifier-checkable tasks are chosen to isolate persistence from the confound that an agent may simply be unable to solve an open-ended unit.

The task manifests are constructed before evaluation and define the repository, target count, budget, objective text, and hidden verifier state. In QGP-RepoScan, valid identifiers are generated offline from stable repository artifact IDs under the three predicate families; agents receive the objective and search observations, but not the hidden valid-ID set. In QGP-DataOps-lite, each backlog unit contains a public prompt, artifact path, and deterministic checker command; a unit counts only after the checker accepts it. Budgets are fixed by

target: QGP-RepoScan uses 30, 60, 100, and 180 steps for $N = 10, 25, 50, 100$, respectively, while QGP-DataOps-lite uses 30, 50, 90, and 160 steps for $N = 3, 5, 10, 20$. Allowed actions are JSON-structured search, submit, final, and ask-user actions for retrieval; work-unit tasks additionally allow inspection, edit, checker execution, and unit submission.

Agents and models. For QGP-RepoScan, we evaluate three GPT-family models: gpt-4.1-mini, gpt-4.1, and gpt-5.4. Native LLMPolicy and LangGraph are each evaluated under standard, verifier-gated, and STATEQGP controllers, giving 36 task instances with one execution per task instance and controller condition, or 108 executions per backend-model pair and 648 Native/LangGraph executions over fixed task instances. We also evaluate LETTA/MEMGPT and LangGraph+Memory under the standard controller only, because they represent external memory baselines rather than PushBench controller ablations, adding two baselines $\times 36$ task instances $\times 3$ models = 216 executions. For QGP-DataOps-lite, we use the same three-model parity design: Native LLMPolicy and LangGraph use standard, verifier-gated, and UNITQGP controllers, while LETTA/MEMGPT and LangGraph+Memory use the standard controller as external memory baselines. Native and LangGraph are controlled execution backends rather than representative samples of commercial or open-source agent designs. Additional coding-style and frontier-agent studies are discussed in Section 9 and detailed in the appendix.

Evaluation setup. All executions use fixed task manifests, fixed target-dependent budgets, and verifier-derived success labels. Within each task family, controller comparisons use the same task, budget, verifier, and valid set; only the controller’s exposure of progress state, duplicate filtering, completion gating, and repair routing changes. Detailed implementation artifacts and reliability metrics are included for reproducibility.

Metrics. We report target success rate, average verified valid count, duplicate submit rate, and valid artifacts per step. Success needs $\text{valid_count} \geq \text{target_count}$ and is evaluated from verifier state not from the agent’s final message.

7 Experiment 1: Repository-Scanning Results

Table 1 reports the QGP-RepoScan results. Each row aggregates one execution on each of 36 fixed task instances: nine task instances at each target count $N \in \{10, 25, 50, 100\}$. Appendix Figure 3 decomposes these aggregate rows by target count.

Main finding. Standard and verifier-gated agents improve with stronger models, showing that QGP-RepoScan is not impossible for non-persistent controllers. However, the same QGP failure mode remains visible: standard and verifier-gated rows still degrade sharply on larger targets and retain duplicate or premature-stop behavior. For example, gpt-5.4 Native standard reaches 30.6% success and a much lower duplicate rate than smaller models, but gpt-5.4 LangGraph standard reaches only 16.7% success with a 0.663 duplicate submit rate. Verifier gating blocks unsupported termination, yet it does not add the duplicate-aware progress state required to make large target counts reliable.

By contrast, STATEQGP changes the execution profile across all three models and both agent implementations. It reaches 72.2% success for gpt-4.1-mini with both Native and LangGraph, 72.2% for gpt-4.1 with both backends, 69.4% for gpt-5.4 Native, and 77.8% for gpt-5.4 LangGraph. It also keeps duplicate submit rate at 0.000 in every row. Because tasks, budgets, verifiers, and valid sets are matched, this improvement isolates the value of controller-visible progress state rather than a change in the underlying task.

Target scaling. The target-count breakdown shows that small-target success does not imply persistence at larger targets. Many standard and verifier-gated rows succeed at $N = 10$, but success often drops to zero by $N = 50$ or $N = 100$. STATEQGP remains strongest at small and medium targets and preserves zero duplicate submissions across all targets, although high-target conditions can still exhaust the budget. This is the intended interpretation: controller persistence reduces repeated work and premature completion, but it does not make every long-horizon retrieval target free. Appendix B.2, Figure 3, reports the same pattern against memory baselines: memory can reduce duplicates, but its high-target success is model-dependent, while STATEQGP preserves the most consistent target scaling.

External memory baselines. LETTA/MEMGPT and LangGraph+Memory test whether run-local memory is enough to maintain progress without a QGP-specific controller. The results show that it helps for stronger models, but the effect is not consistent across models. With gpt-4.1-mini, both memory baselines remain far below STATEQGP: LETTA/MEMGPT completes 0 of 36 task instances and LangGraph+Memory completes 2 of 36, while STATEQGP completes 72.2% for both Native and LangGraph. With gpt-4.1, memory improves substantially: LETTA/MEMGPT reaches 44.4% and LangGraph+Memory reaches 47.2%, but both remain below the 72.2% STATEQGP rows. With gpt-5.4, LETTA/MEMGPT and LangGraph+Memory both reach 72.2%, competitive with Native STATEQGP at 69.4% and close to LangGraph STATEQGP at 77.8%. Thus memory can help with stronger policies and sometimes approaches controller-level persistence, but it is not a uniform replacement across model scales. Appendix B.1 reports the detailed breakdown.

8 Experiment 2: Work-Unit Results

QGP-DataOps-lite asks whether the same persistence pattern survives when units are no longer just retrieved identifiers. QGP-RepoScan isolates counting and retrieval over repository artifacts; QGP-DataOps-lite tests whether the same state problem appears when each unit must be inspected, checked, edited or answered, verified, and submitted. QGP-DataOps-lite uses public data snippets and repository-derived fixtures, but still keeps unit difficulty low enough that the experiment is mainly about persistence, state fidelity, and recovery rather than open-ended software engineering.

Figure 2 reports the QGP-DataOps-lite comparison. Each cell aggregates one execution on each of 24 fixed backlogs for one model: six backlogs at each target count $N \in \{3, 5, 10, 20\}$ for each controller or memory baseline.

Main finding. Standard and verifier-gated controllers finish no QGP-DataOps-lite task instances across all three models and both controlled backends. This strengthens the work-unit conclusion: once tasks require inspection, checking, small edits or answers, and verifier-backed submission, stronger policies alone do not solve the quantitative stopping problem under passive execution. Verifier gating prevents unsupported completion, but lacks the progress state needed to finish the backlog.

Model	Agent	Controller	Task inst.	Succ.	Avg. valid	Dup. rate	Valid/step
gpt-4.1-mini	Native	Standard	36	0.028	6.42	0.880	0.103
gpt-4.1-mini	Native	Verifier-gated	36	0.083	6.64	0.872	0.128
gpt-4.1-mini	Native	STATEQGP	36	0.722	38.67	0.000	2.229
gpt-4.1-mini	LangGraph	Standard	36	0.083	5.36	0.868	0.106
gpt-4.1-mini	LangGraph	Verifier-gated	36	0.028	6.44	0.881	0.103
gpt-4.1-mini	LangGraph	STATEQGP	36	0.722	37.94	0.000	2.127
gpt-4.1-mini	LETTA/MEMGPT	Standard	36	0.000	2.39	0.129	0.179
gpt-4.1-mini	LG+Memory	Standard	36	0.056	2.25	0.468	0.389
gpt-4.1	Native	Standard	36	0.306	17.11	0.557	0.615
gpt-4.1	Native	Verifier-gated	36	0.333	18.64	0.640	0.692
gpt-4.1	Native	STATEQGP	36	0.722	38.53	0.000	2.276
gpt-4.1	LangGraph	Standard	36	0.306	17.78	0.681	0.700
gpt-4.1	LangGraph	Verifier-gated	36	0.389	19.33	0.747	0.589
gpt-4.1	LangGraph	STATEQGP	36	0.722	38.92	0.000	2.328
gpt-4.1	LETTA/MEMGPT	Standard	36	0.444	31.83	0.033	0.412
gpt-4.1	LG+Memory	Standard	36	0.472	31.86	0.006	3.117
gpt-5.4	Native	Standard	36	0.306	20.89	0.112	1.177
gpt-5.4	Native	Verifier-gated	36	0.472	26.47	0.216	0.804
gpt-5.4	Native	STATEQGP	36	0.694	37.00	0.000	2.337
gpt-5.4	LangGraph	Standard	36	0.167	11.83	0.663	0.480
gpt-5.4	LangGraph	Verifier-gated	36	0.222	14.81	0.743	0.374
gpt-5.4	LangGraph	STATEQGP	36	0.778	41.75	0.000	2.537
gpt-5.4	LETTA/MEMGPT	Standard	36	0.722	37.25	0.009	1.718
gpt-5.4	LG+Memory	Standard	36	0.722	36.69	0.005	3.353

Table 1: QGP-RepoScan results. Each row aggregates 36 fixed real-project artifact-finding task instances over requests, pytest, and flask: nine task instances at each target count 10, 25, 50, and 100. Appendix Figure 3 reports the per-target breakdown. Native and LangGraph rows are matched controller comparisons; LETTA/MEMGPT and LG+Memory are standard-controller external memory baselines. All values are from verifier-visible state.

UNITQGP restores nonzero completion in every model-backend pair: 50.0% for gpt-4.1-mini, 29.2% for gpt-4.1, and 25.0% for gpt-5.4 under both Native and LangGraph. The absolute success rates are lower than QGP-RepoScan, as expected for verifier-backed work units, but the controller contrast is cleaner: the matched standard and verifier-gated rows are all zero.

Memory baselines. Memory baselines are useful but model-dependent. LETTA/MEMGPT reaches 20.8%, 50.0%, and 45.8% success from gpt-4.1-mini to gpt-5.4, while LangGraph+Memory reaches 4.2%, 8.3%, and 41.7%. This mirrors the QGP-RepoScan pattern: generic memory can be competitive with a strong policy, but it is not a uniform substitute for verifier-aligned progress state. We make the work-unit conclusion conservatively: controller-level UNITQGP is one effective implementation of verifier-aligned state, while sufficiently strong memory systems can sometimes recover comparable behavior.

Failure analysis. Inspection of representative executions exposes several distinct failure modes. Some standard executions claim completion with more units than the verifier accepts, while others stop early or exhaust budget without making enough accepted submissions. Verifier gating addresses unsupported final claims but not no-submit or stale-inspection loops. In UNITQGP executions, the controller repairs stale inspection, missing verifier calls, and no-submit loops by steering the agent back toward pending units and checks. These repairs are controller-visible interventions, not hidden model improvements, and executions that hit conservative no-progress limits are still classified as budget-exhausted. Appendix C reports full controller metrics, paired controller deltas, and a unit-kind breakdown for this evaluation.

9 Discussion and Additional Evaluations

QGP as operational reliability. The experiments show that many failures are not zero-

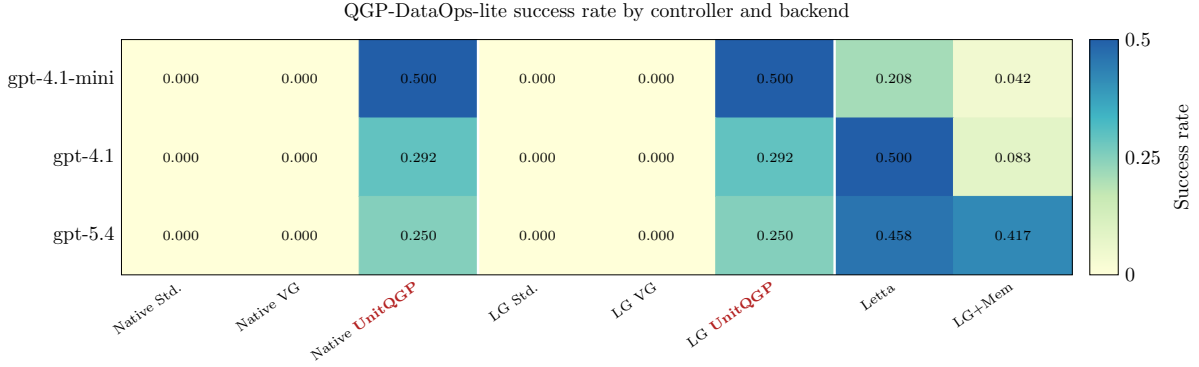


Figure 2: QGP-DataOps-lite success rates. Native and LangGraph (LG) use standard, verifier-gated (VG), and UNITQGP controllers. LETTA/MEMGPT and LangGraph+Memory (LG+Mem) are standard-controller memory baselines. The heatmap emphasizes that passive and completion-gated controllers complete no task instances, while backlog-tracking and memory-based variants recover nonzero completion.

progress failures. Agents often find or complete valid units, but lose the run by repeating accepted work, stopping before the target count, or failing to use verifier feedback to choose the next unit. QGP makes this gap measurable: success requires accumulated verifier-accepted progress, not a plausible final claim or a handful of correct local steps. This is why the same failure pattern appears in both QGP-RepoScan and QGP-DataOps-lite.

Controllers as visible interventions. The controller comparisons show that progress state is not just bookkeeping around the agent; it changes the trajectory. Passive controllers leave the policy to remember which units were submitted, which were accepted, and whether final completion is justified. State-tracking controllers expose that state at each step and use it to block unsupported final actions, filter duplicates, advance repeated searches, and route work-unit agents back to pending checks. Since the verifier, task instances, and budgets are fixed, the gains point to a runtime failure mode: without visible progress state, agents can spend the budget repeating work they have done or ending before the count invariant is met. Appendix B.3 decomposes the STATEQGP components.

Memory and coding-style evaluations. The extra evaluations enhance the same conclusion under less controlled conditions: QGP failures are reduced by stronger infrastructure, but they do not disappear as a measurement problem. The memory baselines show that generic run-local memory can help, especially with stronger policies, yet the gains are model-dependent and do not uniformly replace verifier-aligned progress state. In

the coding-style QGP-SWE-lite evaluation, passive and verifier-gated Native/LangGraph agents complete no task instances, while UNITQGP recovers 37.5–41.7% success. Aider (Paul Gauthier, 2026) succeeds on most task instances, indicating that the units are feasible, but also that coding-agent harness behavior changes the problem being measured. Appendix D.1 reports the detailed breakdown.

Frontier agents. The black-box evaluation provides a complementary check with stronger coding-agent products: they solve many smaller-count cases, but the larger explicit-count goal still exposes persistence failures. Claude Code with Sonnet 4.6 and Codex CLI with gpt-5.4 solve many 50-artifact QGP-RepoScan cases, but each condition drops to 3/9 successes at 100 artifacts. Adding an explicit checklist prompt does not improve paired success, suggesting that generic reminders are weaker than verifier-aligned progress tracking. Appendix D.2 and Figure 5 report the details.

10 Conclusion

We introduced Quantitative Goal Persistence, a formulation for evaluating whether long-horizon agents persist until an explicit quantitative completion invariant is externally satisfied. Our experiments suggest that high-count persistence remains nontrivial even for stronger black-box frontier-agent harnesses. We hope QGP-style metrics become a standard part of agent evaluation, complementing end-to-end task success with a direct measure of whether agents preserve verified progress until the requested work is actually complete.

11 Limitations

This work has five main limitations.

1. First, QGP-RepoScan and QGP-DataOps-lite are controlled QGP benchmarks, not general software-agent leaderboards. The included models test whether persistence failures remain visible under stronger policies. They are designed to isolate quantitative persistence under controlled conditions, not to rank general-purpose agents or estimate their overall software-engineering ability.
2. Second, the task coverage is still limited. QGP-RepoScan uses three repositories and 36 task instances, while QGP-DataOps-lite uses public data snippets and repository-derived fixtures. Broader repositories, data sources, and task types are needed before making claims about software or data-agent workloads in general.
3. Third, the external memory baselines are practical comparisons rather than controlled ablations. Systems such as LETTA/MEMGPT and LangGraph+Memory include their own runtime and memory semantics, so their results should be read as evidence about whether generic memory helps, not as isolated tests of a single controller component.
4. Fourth, the coding-style and black-box frontier-agent evaluations trade control for realism. Coding tasks mix persistence with coding skill, repository navigation, editing ability, and harness behavior. Frontier-agent products add hidden prompting, memory, tool routing, retry behavior, and product updates. These evaluations therefore provide external-validity evidence, but they do not establish general weaknesses of commercial or open-source agents.
5. Finally, STATEQGP and UNITQGP require an online verifier or checker. The controller does not need access to the hidden valid set, but it does need feedback on submitted units during execution. The approach does not directly apply to tasks where progress can only be judged at the end by a human. When explicit count goals are only one part of success, QGP should also be complemented with qualitative evaluation.

12 Ethical Considerations

PushBench is intended as an evaluation framework for agent reliability. The main ethical risk is overinterpreting small controlled pilots as general claims about model safety or competence. We mitigate this by making controller assistance explicit and by reserving numeric claims for verifier-derived metrics. Software repository and public-data sources are cited in the benchmark descriptions.

13 Open Data and Artifacts

An anonymized PushBench artifact is available at <https://anonymous.4open.science/r/artifacts-pushbench-7DB1/README.md>. It includes the code, manifests, repository snapshots, aggregate summaries, table data, and reproduction scripts needed to regenerate the reported tables and run verifier smoke checks. The artifact excludes raw model traces, private endpoint configuration, and non-anonymized local paths.

References

- Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. [Program synthesis with large language models](#). *CoRR*, abs/2108.07732.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. [Evaluating large language models trained on code](#). *CoRR*, abs/2107.03374.
- Zhiyuan Chen, Shiqi Shen, Guangyao Shen, Gong Zhi, Xu Chen, and Yankai Lin. 2024. [Towards tool use alignment of large language models](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 1382–1400. Association for Computational Linguistics.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. [Mind2web: Towards a generalist agent for the web](#). In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- FiveThirtyEight. 2026. [Fivethirtyeight data: Airline safety](#). <https://github.com/fivethirtyeight/data/blob/master/airline-safety/airline-safety.csv>. Dataset source. Accessed: 2026-05-10.

- Yu Gu, Yiheng Shu, Hao Yu, Xiao Liu, Yuxiao Dong, Jie Tang, Jayanth Srinivasa, Hugo Latapie, and Yu Su. 2024. [Middleware for llms: Tools are instrumental for language agents in complex environments](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 7646–7663. Association for Computational Linguistics.
- Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, Huan Sun, and Yu Su. 2025. [Is your LLM secretly a world model of the internet? model-based planning for web agents](#). *Trans. Mach. Learn. Res.*, 2025.
- Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. [Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models](#). In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Findings of ACL, pages 11143–11156. Association for Computational Linguistics.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. [Measuring coding challenge competence with APPS](#). In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. [Swe-bench: Can language models resolve real-world github issues?](#) In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. [Api-bank: A benchmark for tool-augmented llms](#). *CoRR*, abs/2304.08244.
- Tianyang Liu, Canwen Xu, and Julian J. McAuley. 2024. [Repobench: Benchmarking repository-level code auto-completion systems](#). In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2023. [Agentbench: Evaluating llms as agents](#). volume abs/2308.03688.
- Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, and 3 others. 2021. [Codexglue: A machine learning benchmark dataset for code understanding and generation](#). In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024a. [Agentboard: An analytical evaluation board of multi-turn LLM agents](#). In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024b. [Agentboard: An analytical evaluation board of multi-turn LLM agents](#). In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Ruiying Ma, Shreya Shankar, Ruiqi Chen, Yiming Lin, Sepanta Zeighami, Rajoshi Ghosh, Abhinav Gupta, Anushrut Gupta, Tanmai Gopal, and Aditya G. Parameswaran. 2026. [Can AI agents answer your data questions? A benchmark for data agents](#). *CoRR*, abs/2603.20576.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. [Self-refine: Iterative refinement with self-feedback](#). In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. [Memgpt: Towards llms as operating systems](#). *CoRR*, abs/2310.08560.
- Pallets. 2026. [Flask](#). <https://github.com/pallets/flask>. Software repository. Accessed: 2026-05-10.
- Joon Sung Park, Joseph C. O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. [Generative agents: Interactive simulators of human behavior](#). In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 2023, San Francisco, CA, USA, 29 October 2023- 1 November 2023*, pages 2:1–2:22. ACM.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. [Gorilla: Large language](#)

- model connected with massive apis. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Paul Gauthier. 2026. *Aider*. <https://github.com/Aider-AI/aider>. Software repository. Accessed: 2026-05-10.
- pytest-dev. 2026. *pytest*. <https://github.com/pytest-dev/pytest>. Software repository. Accessed: 2026-05-10.
- Python Software Foundation. 2026. *Requests*. <https://github.com/psf/requests>. Software repository. Accessed: 2026-05-10.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. *Toolllm: Facilitating large language models to master 16000+ real-world apis*. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. *Toolformer: Language models can teach themselves to use tools*. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. *Reflexion: language agents with verbal reinforcement learning*. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew J. Hausknecht. 2021. *Alfworld: Aligning text and embodied environments for interactive learning*. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.
- Vega Project. 2026. *Vega datasets: Cars*. <https://github.com/vega/vega-datasets/blob/main/data/cars.json>. Dataset source. Accessed: 2026-05-10.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024a. *Voyager: An open-ended embodied agent with large language models*. *Trans. Mach. Learn. Res.*, 2024.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024b. *A survey on large language model based autonomous agents*. *Frontiers Comput. Sci.*, 18(6):186345.
- Ruoyao Wang, Peter A. Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. 2022. *Scienceworld: Is your agent smarter than a 5th grader?* In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 11279–11298. Association for Computational Linguistics.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. *Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments*. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. *Swe-agent: Agent-computer interfaces enable automated software engineering*. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. *Intercode: Standardizing and benchmarking interactive coding with execution feedback*. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. *Webshop: Towards scalable real-world web interaction with grounded language agents*. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. *τ -bench: A benchmark for tool-agent-user interaction in real-world domains*. *CoRR*, abs/2406.12045.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. *React: Synergizing reasoning and acting in language models*. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.
- Junjie Ye, Zhengyin Du, Xuesong Yao, Weijian Lin, Yufei Xu, Zehui Chen, Zaiyuan Wang, Sining Zhu,

Zhiheng Xi, Siyu Yuan, Tao Gui, Qi Zhang, Xuanjing Huang, and Jiecao Chen. 2025. [Toolhop: A query-driven benchmark for evaluating large language models in multi-hop tool use](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, pages 2995–3021. Association for Computational Linguistics.

Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. [Assistantbench: Can web agents solve realistic and time-consuming tasks?](#) In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 8938–8968. Association for Computational Linguistics.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. [Webarena: A realistic web environment for building autonomous agents](#). In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

The appendix gives more details of the evaluation. Appendix A gives task-construction details for PushBench. Appendix B reports additional QGP-RepoScan memory, target-scaling, and ablation analyses. Appendix C expands the QGP-DataOps-lite results. Appendix D contains coding-style and frontier-agent evaluations. All appendix results use the same verifier-derived metrics as the experiments above.

A PushBench Benchmark Details

QGP-RepoScan task instances vary by repository, target count, seed, and predicate family. Artifacts may be source, test, documentation, or configuration items, and predicate families include keyword-or-pattern, path-and-content, and test-or-documentation membership. Agents see the objective and search observations, but the valid identifier set remains hidden inside the verifier.

QGP-DataOps-lite backlogs combine public data snippets and repository-derived fixtures. Public snippets come from Vega cars data (Vega Project, 2026) and FiveThirtyEight airline-safety data (FiveThirtyEight, 2026); repository fixtures are derived from the same local requests, pytest, and flask snapshots used by QGP-RepoScan. They are small, deterministic tasks over repository files or metadata rather than open-ended code changes. Units include CSV schema or count

checks, consistency-check answers, metadata repairs, deterministic verifier execution, and submission of the accepted unit. These units are intended to require concrete checking and recovery without making the underlying work open-ended.

Additional task variants. The same search-submit-verify interface can be instantiated for synthetic retrieval corpora and code-location retrieval over repository text chunks. The reported claims use the QGP-RepoScan and QGP-DataOps-lite experiments, plus the coding-style and frontier-agent evaluations in Appendix D.

B Additional RepoScan Analyses

B.1 Memory Baseline Breakdown

Table 2 expands the memory rows in Table 1 with failure-mode counts that are not shown in the main result table. The LangGraph+Memory baseline uses the same 36 QGP-RepoScan tasks and budgets, uses only the standard controller, and records previous compact observations, status snapshots, actions, submitted identifiers, searched pages, and parse errors. It does not receive hidden valid identifiers or verifier labels.

LangGraph+Memory is model-dependent. With gpt-4.1-mini, it does not improve success over plain LangGraph and remains far below STATE-QGP. With gpt-4.1, it improves standard LangGraph from 30.6% to 47.2% success and nearly eliminates duplicate submissions, but still trails LangGraph STATEQGP at 72.2%. With gpt-5.4, LangGraph+Memory reaches 72.2%, matching LETTA/MEMGPT and approaching LangGraph STATEQGP at 77.8%. The pattern refines the memory comparison: memory can be a viable implementation strategy when paired with a strong policy, while controller-level persistence remains a simple and consistently strong way to expose and enforce the QGP state contract.

B.2 Target Scaling and Paired Uncertainty

Figure 3 decomposes the aggregate QGP-RepoScan rows from Table 1 by target count for LangGraph, memory baselines, and STATEQGP. Each target bucket contains the same nine fixed task instances used in Table 1. The central pattern is target scaling rather than small-target success: many rows solve $N = 10$ but collapse by $N = 50$ or $N = 100$, while STATEQGP remains nonzero at the largest target for all three models.

Model	Baseline	Succ.	Avg. valid	Dup. rate	Premature	Budget exh.	Valid/step
gpt-4.1-mini	LangGraph Standard	0.083	5.36	0.918	0.000	0.917	0.060
gpt-4.1-mini	LangGraph+Memory	0.056	2.25	0.468	0.583	0.000	0.389
gpt-4.1-mini	LangGraph STATEQGP	0.722	37.94	0.000	0.000	0.278	0.781
gpt-4.1-mini	LETTA/MEMGPT Standard	0.000	2.39	0.159	0.889	0.111	0.169
gpt-4.1	LangGraph Standard	0.306	17.78	0.858	0.083	0.611	0.385
gpt-4.1	LangGraph+Memory	0.472	31.86	0.006	0.111	0.000	3.117
gpt-4.1	LangGraph STATEQGP	0.722	38.92	0.000	0.000	0.278	0.820
gpt-4.1	LETTA/MEMGPT Standard	0.444	31.83	0.062	0.111	0.444	0.392
gpt-5.4	LangGraph Standard	0.167	11.83	0.867	0.583	0.250	0.274
gpt-5.4	LangGraph+Memory	0.722	36.69	0.005	0.278	0.000	3.353
gpt-5.4	LangGraph STATEQGP	0.778	41.75	0.000	0.000	0.222	0.977
gpt-5.4	LETTA/MEMGPT Standard	0.722	37.25	0.017	0.278	0.000	1.492

Table 2: QGP-RepoScan memory baseline breakdown. Generic memory can sharply reduce duplicate submissions, especially with stronger models, but success and stopping behavior remain model-dependent.

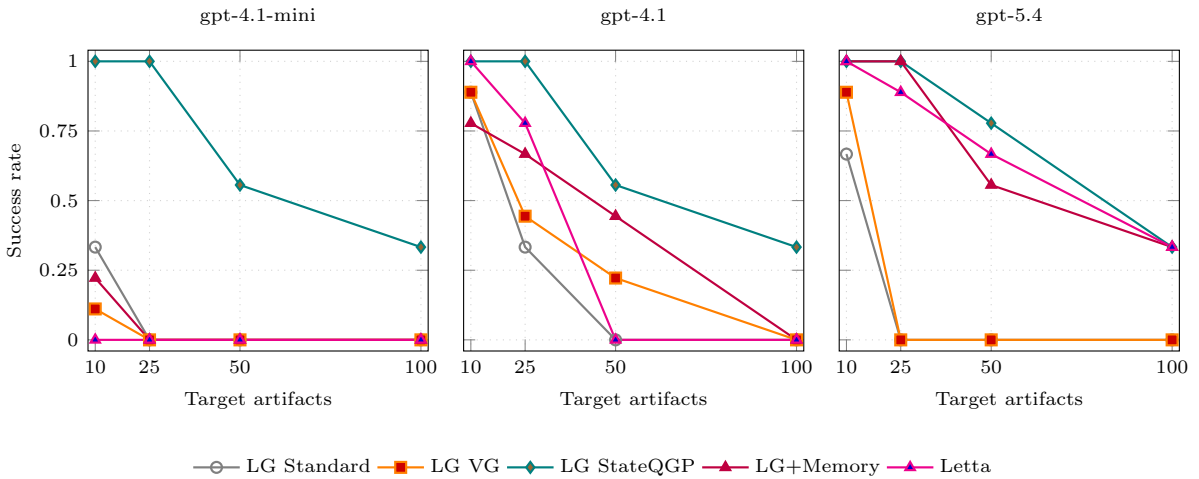


Figure 3: QGP-RepoScan target scaling. Each point aggregates the nine task instances in one target bucket from Table 1. Many baselines solve small targets but collapse as the requested artifact count grows; STATEQGP preserves nonzero high-target success while maintaining zero duplicate submissions.

A paired bootstrap over matched task instances gives the same picture. For each bootstrap sample, we resample task instances with replacement and recompute the success-rate difference between two controllers on the same sampled tasks. The reported interval is therefore an uncertainty range for the difference, not for the success rate itself. On gpt-4.1 LangGraph, STATEQGP raises success over the standard controller by 41.7 percentage points, with a 95% interval from 27.8 to 58.3 points. Verifier gating raises success by only 8.3 points, and its interval ranges from -2.8 to 22.2 points; the negative lower end means that, under some resampled task sets, verifier gating does not improve over the standard controller. For gpt-5.4 LangGraph, the STATEQGP gain over standard is 61.1 points, with a 95% interval from 44.4 to 77.8

points.

B.3 Stateful Component Ablation

We use a focused gpt-4.1 LangGraph ablation to separate the main components of STATEQGP. The ablation reuses the same 36 QGP-RepoScan tasks and adds three variants: duplicate filtering only, repeated-search page memory only, and duplicate filtering plus page memory without buffered seen-document submission. Figure 4 compares these variants against the standard, verifier-gated, and full STATEQGP rows, which are the matched QGP-RepoScan rows for the same model and backend.

The ablation shows that no single passive memory feature explains the STATEQGP result. Duplicate filtering eliminates duplicate submissions but does not improve success beyond verifier gating.

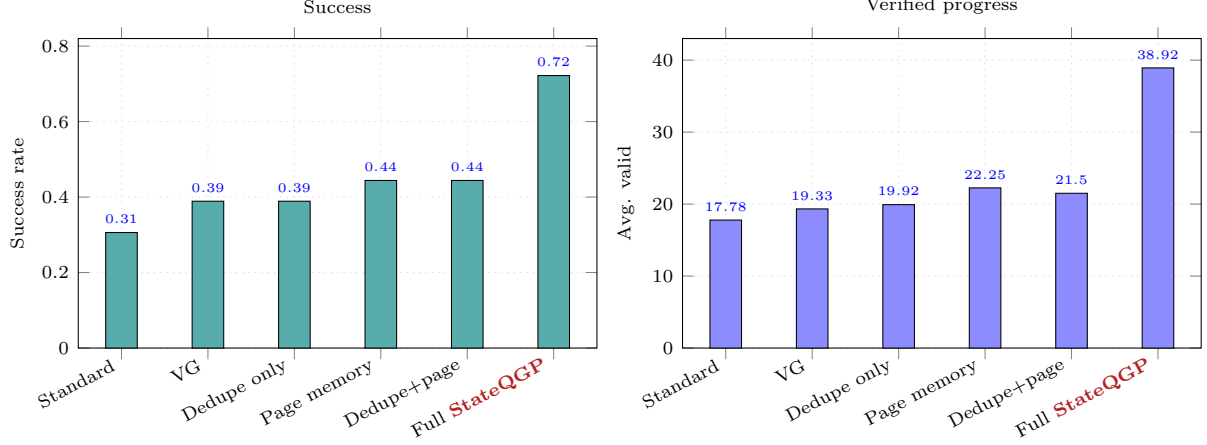


Figure 4: Focused gpt-4.1 LangGraph stateful ablation. Success and average verified count rise most sharply only when duplicate filtering, page memory, and buffered verifier-aligned progress state are combined in full STATEQGP.

Page memory improves success to 44.4%, but the full controller reaches 72.2%. The paired success delta between full STATEQGP and the no-buffer dedupe+page variant is 0.278 with a 95% bootstrap interval of [0.139, 0.417]. The key mechanism is therefore not generic memory alone, but verifier-aligned progress state that can turn seen candidates into forward progress.

C DataOps-lite Detailed Results

Table 3 reports the full QGP-DataOps-lite controller metrics behind Figure 2. Standard and verifier-gated failures are dominated by budget exhaustion, premature stopping, and very low accepted-unit counts. Table 4 gives paired controller deltas over matched tasks. UNITQGP improves success over both standard and verifier-gated controllers for every model-backend pair, while verifier gating alone has zero success delta over standard. Table 5 groups executions by unit kind. UNITQGP is strongest on artifact-validation units and weaker on consistency-check and metadata-update units; memory-baseline successes concentrate in mixed backlogs, which helps rule out a single retrieval-style shortcut.

D External-Validity Evaluations

D.1 SWE-lite Realism Evaluation

QGP-SWE-lite is a coding-style extension of Push-Bench inspired by software-engineering agent benchmarks such as SWE-bench and SWE-agent (Jimenez et al., 2024; Yang et al., 2024). Instead of asking the agent to retrieve identifiers, each valid unit is an issue-style repair or check that must

pass a local verifier. The units are constructed from isolated real-workspace copies of requests, flask, and pytest (Python Software Foundation, 2026; Pallets, 2026; pytest-dev, 2026). Targets are $N \in \{1, 2, 3, 5\}$, with six repo/seed conditions per target, for 192 completed executions across Native LLMPolicy, LangGraph, and Aider (Paul Gauthier, 2026). These results are a coding-style external-validity check because the setting entangles persistence with coding skill, repository navigation, editing ability, and harness behavior.

Native and LangGraph standard and verifier-gated controllers complete no QGP-SWE-lite task instances. Verifier gating again prevents unsupported completion from ending the run, but it does not repair empty search loops, stale inspection, missing unit submissions, or failure to run the verifier after edits. UNITQGP materially changes the outcome for both implementations, reaching 37.5% success for Native and 41.7% for LangGraph, with average verified counts rising from 0.00 to 1.25 and 1.46 respectively.

Aider provides a black-box solvability contrast. Its 91.7% success rate under both standard and UNITQGP execution shows that the QGP-SWE-lite units are feasible under the model and verifier budget. The contrast also illustrates why coding-agent harnesses are reported separately: their editing loop changes more than the QGP controller.

Black-box agent caveat. Commercial or product-style coding agents can be evaluated with the same verifier-facing submission interface. Hidden prompting, memory, tool routing, retry behavior, and product updates make their internal

Model	Backend	Controller	Backlogs	Succ.	Avg. valid	Dup. rate	Valid/step	Budget exh.	Premature	False comp.
gpt-4.1-mini	Native	Standard	24	0.000	0.708	0.125	0.036	0.833	0.167	0.000
gpt-4.1-mini	Native	VG	24	0.000	0.833	0.083	0.037	1.000	0.000	0.000
gpt-4.1-mini	Native	UNITQGP	24	0.500	8.000	0.059	0.136	0.500	0.000	0.000
gpt-4.1-mini	LangGraph	Standard	24	0.000	0.708	0.050	0.046	0.917	0.083	0.000
gpt-4.1-mini	LangGraph	VG	24	0.000	0.792	0.042	0.042	1.000	0.000	0.000
gpt-4.1-mini	LangGraph	UNITQGP	24	0.500	8.042	0.101	0.144	0.500	0.000	0.000
gpt-4.1-mini	LETTA/MEMGPT	Standard	24	0.208	4.917	0.044	0.242	0.000	0.792	0.000
gpt-4.1-mini	LG+Mem	Standard	24	0.042	2.250	0.253	0.054	0.208	0.583	0.167
gpt-4.1	Native	Standard	24	0.000	0.875	0.483	0.046	0.792	0.208	0.000
gpt-4.1	Native	VG	24	0.000	2.417	0.216	0.089	1.000	0.000	0.000
gpt-4.1	Native	UNITQGP	24	0.292	7.333	0.569	0.102	0.708	0.000	0.000
gpt-4.1	LangGraph	Standard	24	0.000	1.125	0.558	0.057	0.792	0.208	0.000
gpt-4.1	LangGraph	VG	24	0.000	2.625	0.427	0.069	1.000	0.000	0.000
gpt-4.1	LangGraph	UNITQGP	24	0.292	7.167	0.604	0.098	0.708	0.000	0.000
gpt-4.1	LETTA/MEMGPT	Standard	24	0.500	7.500	0.049	0.245	0.000	0.500	0.000
gpt-4.1	LG+Mem	Standard	24	0.083	5.833	0.013	0.190	0.250	0.500	0.167
gpt-5.4	Native	Standard	24	0.000	0.000	0.000	0.000	0.542	0.458	0.000
gpt-5.4	Native	VG	24	0.000	0.000	0.000	0.000	1.000	0.000	0.000
gpt-5.4	Native	UNITQGP	24	0.250	6.708	0.000	0.094	0.750	0.000	0.000
gpt-5.4	LangGraph	Standard	24	0.000	0.000	0.000	0.000	1.000	0.000	0.000
gpt-5.4	LangGraph	VG	24	0.000	0.000	0.000	0.000	1.000	0.000	0.000
gpt-5.4	LangGraph	UNITQGP	24	0.250	6.542	0.060	0.092	0.750	0.000	0.000
gpt-5.4	LETTA/MEMGPT	Standard	24	0.458	7.042	0.040	0.290	0.000	0.542	0.000
gpt-5.4	LG+Mem	Standard	24	0.417	7.208	0.018	0.299	0.375	0.208	0.000

Table 3: QGP-DataOps-lite full controller metrics. Each row aggregates 24 fixed backlogs, with six at each target count. Provider-error rate is 0.000 for every row in the evaluation.

behavior only partially observable, so these evaluations complement rather than replace the controlled controller comparisons.

D.2 Black-Box Frontier-Agent RepoScan Evaluation

To test whether stronger commercial or frontier agent products still expose QGP failures, we evaluate QGP-RepoScan at the hardest target counts. The evaluation uses the 18 QGP-RepoScan tasks with $N \in \{50, 100\}$, two black-box harnesses, and two prompt modes. The default prompt asks the agent to continue until the verifier status reaches the target. The checklist prompt adds explicit instructions to track accepted identifiers, rejected or duplicate identifiers, searched query/page pairs, remaining count, and final eligibility. Agents interact only through a thin local tool exposing search, submit, status, and final; hidden valid identifiers are kept inside the verifier process. Claude Code is invoked with `-model sonnet` and reported as Claude Code Sonnet 4.6 (claude-sonnet-4-6). Codex CLI is invoked with `-m gpt-5.4`.

Agent	N	Pairs	Succ.	Δ	Avg-valid Δ	Dup. Δ
Claude Code (Sonnet 4.6)	50	9	0.000		1.00	2.67
Claude Code (Sonnet 4.6)	100	9	0.000		-0.44	13.44
Codex CLI (gpt-5.4)	50	9	0.000		-1.67	0.22
Codex CLI (gpt-5.4)	100	9	0.000		0.44	-8.78

Table 7: Checklist-minus-default paired deltas over matched task and agent. The checklist prompt changes duplicate and tool-use behavior but does not improve success in this evaluation.

Figure 5 shows that frontier-agent performance improves the absolute success rate without eliminating the high-count persistence problem. Both black-box agents solve seven of nine $N = 50$ tasks under both prompts, showing that the search-and-submit interface and task construction are usable. At $N = 100$, however, every agent-prompt condition drops to three of nine successes. The checklist prompt has zero paired success delta for both agents and target counts. Generic progress reminders alone therefore do not guarantee robust high-target performance; the remaining gap points back to mechanisms that track verifier-aligned progress explicitly.

Model	Backend	Delta	Paired tasks	Succ. delta	95% CI	Avg-valid delta	Left-only	Right-only
gpt-4.1-mini	Native	UNITQGP -Std.	24	0.500	[0.292, 0.708]	7.292	12	0
gpt-4.1-mini	Native	UNITQGP -VG	24	0.500	[0.292, 0.708]	7.167	12	0
gpt-4.1-mini	Native	VG-Std.	24	0.000	[0.000, 0.000]	0.125	0	0
gpt-4.1-mini	LangGraph	UNITQGP -Std.	24	0.500	[0.292, 0.708]	7.333	12	0
gpt-4.1-mini	LangGraph	UNITQGP -VG	24	0.500	[0.292, 0.708]	7.250	12	0
gpt-4.1-mini	LangGraph	VG-Std.	24	0.000	[0.000, 0.000]	0.083	0	0
gpt-4.1	Native	UNITQGP -Std.	24	0.292	[0.125, 0.458]	6.458	7	0
gpt-4.1	Native	UNITQGP -VG	24	0.292	[0.125, 0.500]	4.917	7	0
gpt-4.1	Native	VG-Std.	24	0.000	[0.000, 0.000]	1.542	0	0
gpt-4.1	LangGraph	UNITQGP -Std.	24	0.292	[0.125, 0.500]	6.042	7	0
gpt-4.1	LangGraph	UNITQGP -VG	24	0.292	[0.125, 0.500]	4.542	7	0
gpt-4.1	LangGraph	VG-Std.	24	0.000	[0.000, 0.000]	1.500	0	0
gpt-5.4	Native	UNITQGP -Std.	24	0.250	[0.083, 0.417]	6.708	6	0
gpt-5.4	Native	UNITQGP -VG	24	0.250	[0.083, 0.458]	6.708	6	0
gpt-5.4	Native	VG-Std.	24	0.000	[0.000, 0.000]	0.000	0	0
gpt-5.4	LangGraph	UNITQGP -Std.	24	0.250	[0.083, 0.417]	6.542	6	0
gpt-5.4	LangGraph	UNITQGP -VG	24	0.250	[0.083, 0.417]	6.542	6	0
gpt-5.4	LangGraph	VG-Std.	24	0.000	[0.000, 0.000]	0.000	0	0

Table 4: QGP-DataOps-lite paired controller deltas over matched tasks. Left-only counts tasks solved by the left controller but not the right controller; right-only counts the converse. Memory baselines are excluded from this causal controller comparison.

Unit kind	Std./VG	UNITQGP	Memory
Artifact val.	36 / 0.000 / 0.00	18 / 1.000 / 3.67	18 / 0.000 / 0.00
Consistency	24 / 0.000 / 0.00	12 / 0.000 / 0.33	12 / 0.000 / 0.00
Metadata	12 / 0.000 / 0.00	6 / 0.167 / 2.17	6 / 0.000 / 0.00
Mixed	216 / 0.000 / 1.12	108 / 0.287 / 8.96	108 / 0.380 / 7.72

Table 5: QGP-DataOps-lite unit-kind/source breakdown. Each cell reports executions / success rate / average valid units. All unit-kind rows have data_source=mixed, so source breakdown is equivalent to the unit-kind grouping shown here.

Agent	Controller	Conditions	Succ.	Avg. valid	Budget exhausted	Valid/step	Valid/submit
Native	Standard	24	0.000	0.00	1.000	0.000	0.000
Native	Verifier-gated	24	0.000	0.00	1.000	0.000	0.000
Native	UNITQGP	24	0.375	1.25	0.625	0.022	0.250
LangGraph	Standard	24	0.000	0.00	1.000	0.000	0.000
LangGraph	Verifier-gated	24	0.000	0.00	1.000	0.000	0.000
LangGraph	UNITQGP	24	0.417	1.46	0.583	0.027	0.294
Aider	Standard	24	0.917	2.67	0.083	0.173	0.653
Aider	UNITQGP	24	0.917	2.67	0.083	0.176	0.667

Table 6: QGP-SWE-lite realism evaluation results. The benchmark contains issue-style repair units in requests, flask, and pytest. The table reports how QGP evaluation behaves in coding-style workflows.

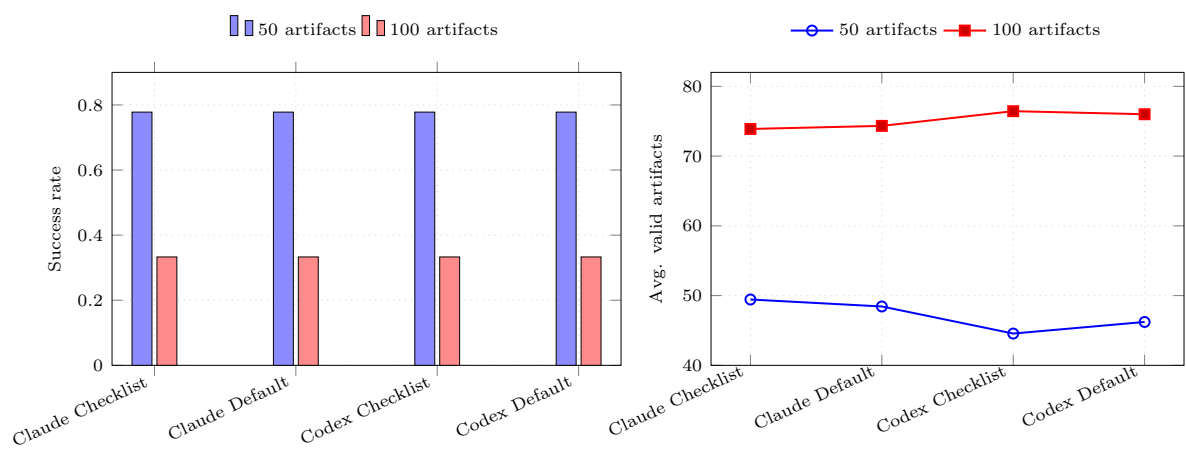


Figure 5: Black-box frontier-agent QGP-RepoScan high-target evaluation. Each condition aggregates nine task instances. Both black-box harnesses solve many 50-artifact tasks, but success drops uniformly at 100 artifacts even when average valid counts remain high.